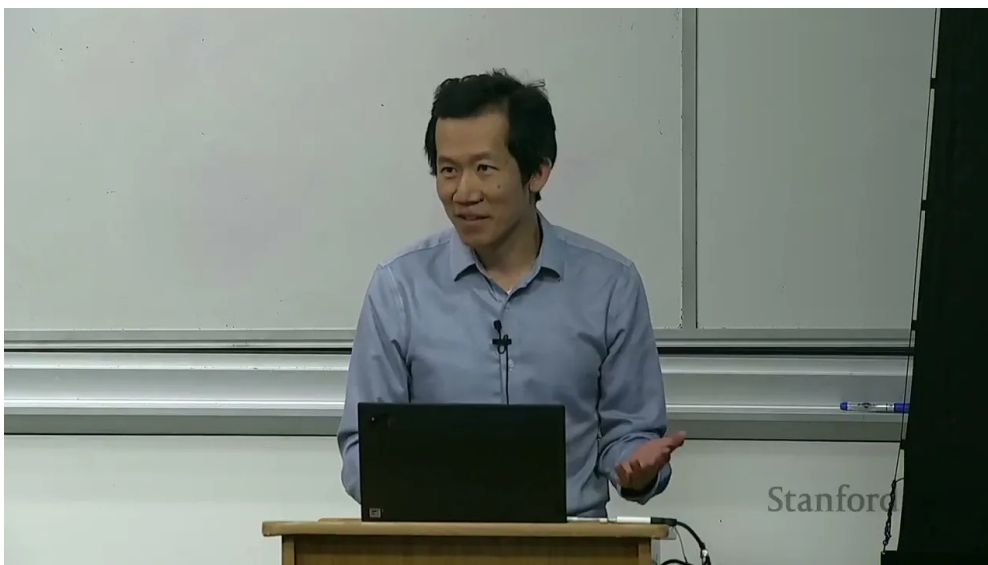


课程笔记

CS336 Lecture 12: Evaluation

基于 Tatsu Hashimoto 授课内容整理

2025 年 5 月



视频作者/频道: Stanford Online

发布日期: 2025-05

视频时长: 1:20:45

视频链接: <https://www.youtube.com/watch?v=>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：为什么评估会变成一门大课	4
1.1 你平时看到的评估长什么样	4
2 如何理解评估	4
2.1 目标先于指标	5
2.2 输入、调用、输出、解释	5
3 Perplexity	5
3.1 为什么 perplexity 仍然重要	5
3.2 Perplexity 与 bits-per-byte 的转换	6
3.3 几种“精神上接近 perplexity”的任务	6
4 Knowledge Benchmarks	7
4.1 MMLU	7
4.2 MMLU 的具体题目示例与评分机制	7
4.3 MMLU-Pro	8
4.4 GPQA 和 Humanity’s Last Exam	8
5 Instruction Following	8
5.1 Chatbot Arena	8
5.2 IFEval	9
5.3 Chatbot Arena ELO 评分的计算直觉	9
5.4 AlpacaEval 和 WildBench	10
5.5 评估方法对比：人工、自动与 LLM-as-Judge	10
5.6 主要 Leaderboard 对比	10
6 Agent Benchmarks	11
6.1 SWE-bench	11
6.2 CyBench	11
6.3 MLEBench	11
7 Pure Reasoning	12
7.1 ARC-AGI	12
7.2 MATH 和 AIME	12
7.3 本章小结	12
8 Safety Benchmarks	12
8.1 HarmBench 与 AIR-Bench	13
8.2 Jailbreaking	13
8.3 Safety 评估中的 taxonomy 问题	13
8.4 Pre-deployment testing	14
8.5 什么是安全	14

9 Realism	14
9.1 Quizzing vs Asking	14
9.2 Clio 和 MedHELM	14
10 Validity	15
10.1 Train-test overlap	15
10.2 Goodhart's Law 与过拟合 Benchmark	16
10.3 Dataset quality	16
10.4 统计显著性: 分差到底够不够大	17
11 我们到底在评估什么	17
11.1 方法还是系统	17
11.2 为什么这很重要	17
11.3 几个例外	17
12 总结	18
13 把评估拆开看: 定义、直觉、机制、应用、局限	18
13.1 定义: 评估到底是什么	18
13.2 直觉: 为什么评估看起来简单, 做起来很难	19
13.3 机制: 一个完整评估管线怎么运作	19
13.4 应用: 什么时候该用什么评估	20
13.5 局限: 为什么没有一个 benchmark 能包打天下	20
14 Worked Example 1: 一个 toy perplexity 计算	20
15 Worked Example 2: 同一模型, 三种评估会给出三种结论	21
16 Worked Example 3: Chatbot Arena 为什么用 pairwise	21
17 Worked Example 4: 安全评估和能力评估为什么会混在一起	22
18 一个更完整的 benchmark 选择指南	23
19 更深一点的局限: 为什么榜单会失真	23
19.1 污染与饱和	23
19.2 奖励错配	23
19.3 judge bias	23
20 Worked Examples: 评估如何真的算出来	24
20.1 例 1: 多任务知识评估怎么汇总	24
20.2 例 2: pairwise 胜率怎么理解	24
20.3 例 3: 安全评估里的混淆矩阵	25
20.4 例 4: 模型研究和产品决策看的是不同东西	25
21 Benchmark Families: 每一类 benchmark 在测什么	25
21.1 主要 Benchmark 详细对比	26

22 如何写一个靠谱的 evaluation report	27
22.1 置信区间不是装饰	27
22.2 一个最小检查清单	28
23 总结与延伸	28
23.1 本章小结	28

1 引言：为什么评估会变成一门大课

这节课讨论 **evaluation**。乍看之下，评估似乎很简单：给定一个固定模型，问它“有多好”，然后算一个分数就行了。实际情况远没有这么直接。评估不仅决定了论文表格里的数字，也在很大程度上决定了模型开发、产品选择、政策判断和安全测试的方向。

本课的中心观点

评估不是一个机械步骤，而是一个需要先定义目标、再定义输入、调用方式、输出度量与结果解释的完整系统设计问题。

课程一开始展示了几个很常见但含义并不完全相同的榜单：模型在 MMLU、MATH、GPQA 等基准上的分数，HELM 的能力榜，Artificial Analysis 的能力-价格前沿，OpenRouter 的流量排行，以及 Chatbot Arena 的人类偏好排名。它们都在回答“模型好不好”，但回答角度完全不同。一个数字本身没有意义，只有它对应的评价目标才有意义。

1.1 你平时看到的评估长什么样

现实中的评估通常包含这些元素：

- **能力基准**：例如 MMLU、MATH、GPQA、Humanity’s Last Exam。
- **偏好排序**：例如 Chatbot Arena 的 pairwise 比较与 ELO。
- **成本视角**：同样的准确率下，推理价格和训练成本可能差很多。
- **使用者视角**：模型不一定要“更聪明”，但一定要“更能卖得出去”或“更适合这个场景”。
- **安全视角**：拒答、越狱、幻觉、双重用途都属于评估范围。

评估并不只看准确率

如果一个模型得分更高，但推理成本高 10 倍、部署风险更大、或者只是在训练数据上记住了测试集，那么它未必是更好的选择。评估必须把成本、鲁棒性和真实使用场景一起考虑进去。

2 如何理解评估

这节课给出的基本框架可以压缩成四个问题：

1. 输入是什么？
2. 如何调用模型？
3. 如何评估输出？
4. 如何解释结果？

2.1 目标先于指标

同一个模型分数，可能对应完全不同的评价目标。

- **用户或公司做购买决策**：该选 Claude、Gemini、Grok，还是别的模型？
- **研究者衡量能力**：模型是否在推进“原始智能”或语言能力？
- **政策和业务分析**：模型带来的收益和危害分别是什么？
- **模型开发者调参**：某个改动是否真的让模型更好？

为什么“我只是评估一下模型”这句话不够

如果不先说清楚目标，评估结果通常会把不同东西混在一起。一个 benchmark 分数可能同时受到 prompt、解码策略、工具使用、训练数据污染、参考答案质量和后处理流程的影响。

2.2 输入、调用、输出、解释

输入部分要问的是：这个评估覆盖了哪些使用场景？有没有尾部难例？输入是不是被改造成更适合模型的形式，比如多轮对话？

调用部分要问的是：是 zero-shot、few-shot，还是 chain-of-thought？有没有工具、RAG 或者其他 agent scaffolding？你评估的是裸模型，还是整个系统？

输出部分要问的是：参考答案是否干净？用什么 metric？像代码任务常用 pass@k，开放式生成可能需要人工偏好、LLM-as-judge 或别的办法。成本要不要算进去？医疗、法律这类场景里，错误是不是有不同代价？

解释部分要问的是：一个 91% 的分数到底意味着什么？能直接上线吗？测试集和训练集有没有重叠？你在评估最终模型，还是某种训练方法？

多轮对话和红队测试会改变输入定义

对于 chat 系统或 red teaming，输入本身往往需要针对模型动态调整。这样更贴近真实行为，但也让不同模型之间的横向对比更难，必须明确规则。

3 Perplexity

在语言模型里，perplexity 是最传统的评估方式之一。把语言模型看成序列分布 $p(x)$ 后，perplexity 可以理解为它给某个数据集 D 赋予的平均困惑程度，常写成

$$\text{PPL}(D) = \left(\frac{1}{p(D)} \right)^{1/|D|}$$

训练时我们最小化训练集上的 perplexity，评估时自然会想看测试集上的 perplexity。

3.1 为什么 perplexity 仍然重要

- 它比 downstream accuracy 更平滑，适合做 scaling law。
- 它是通用的语言建模目标，也是训练目标本身。

- 甚至可以在下游任务上定义条件 perplexity。

perplexity 的局限

如果评估器要信任模型给出的概率分布，那么 perplexity 本身就变成了“要求模型先诚实地报出自己的不确定性”。这比直接看黑盒输出的准确率更脆弱。

3.2 Perplexity 与 bits-per-byte 的转换

在实际论文中，不同团队报告 perplexity 时使用的 tokenizer 不同，导致数值无法直接比较。为了统一，社区越来越倾向于使用 **bits-per-byte** (BPB) 作为与 tokenizer 无关的度量。

转换关系如下。首先，perplexity 和 bits-per-token 的关系是：

$$\text{bits-per-token} = \log_2(\text{PPL})$$

然后，bits-per-byte 需要除以每个 token 平均包含的字节数：

$$\text{bits-per-byte} = \frac{\log_2(\text{PPL})}{\text{avg bytes per token}}$$

Tokenizer	PPL	bits/token	bits/byte (approx)
GPT-2 BPE (avg 3.5 B/tok)	15.0	3.91	1.12
Llama BPE (avg 4.0 B/tok)	8.0	3.0	0.75
字符级 (1 B/tok)	4.5	2.17	2.17

表 1: 不同 tokenizer 下，同样质量的模型 PPL 差异很大，但 bits-per-byte 更具可比性

为什么 bits-per-byte 比 perplexity 更公平

假设模型 A 用的 tokenizer 把“hello”编码成 1 个 token，模型 B 编码成 3 个 token。即使两者给出完全相同的字节级概率分布，模型 B 的 perplexity 也会更低（因为它分更多步做预测，每步更容易）。bits-per-byte 通过归一化到字节级别，消除了这种 tokenizer 差异。

一个具体的换算例子：如果某模型的 perplexity 是 10，使用的 tokenizer 平均每个 token 对应 4 个字节，那么：

$$\text{BPB} = \frac{\log_2(10)}{4} = \frac{3.322}{4} \approx 0.83 \text{ bits/byte}$$

这个数值的直觉含义是：模型平均需要 0.83 个 bit 来编码原始文本中的每个字节。与信息论中英文文本的经验熵（约 1.0–1.5 bits/byte）相比，这意味着模型已经能相当好地预测文本内容。

3.3 几种“精神上接近 perplexity”的任务

课程里提到，某些 cloze 或补全类任务和 perplexity 的思想很接近，比如 LAMBADA、HellaSwag 等。它们本质上仍然是“给定前文，预测后文”，只是形式更贴近下游任务。

最大化 perplexity 的信念

一种极端观点是：如果模型分布 p 最终逼近真实分布 t ，那就能解决所有任务。但这并不意味着对分布的每一部分都同等值得优化，也不意味着它是最有效的通往强模型的路径。

4 Knowledge Benchmarks

知识类 benchmark 是最常见的一类。它们通常采用多选题，测试的是模型是否掌握某些知识，而不一定是真正意义上的“语言理解”。

4.1 MMLU

MMLU 包含 57 个学科，覆盖数学、历史、法律、道德等内容。它原本是为 few-shot prompting 时代设计的，用来粗略衡量模型在广泛学科上的知识覆盖。

- 优点：覆盖面广，形式标准化。
- 缺点：题目质量参差不齐，且越来越容易被过拟合或污染。
- 本质：更像知识测试，不是纯语言理解测试。

4.2 MMLU 的具体题目示例与评分机制

为了让 MMLU 从抽象概念变成具体认知，下面展示一个典型题目（来自 MMLU 的“abstract algebra”子集）：

```
1 Question: Find the degree for the given field
2 extension  $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{18})$ 
3 over  $\mathbb{Q}$ .
4
5 A. 0
6 B. 4
7 C. 2
8 D. 6
9
10 Answer: B
```

Listing 1: MMLU 典型题目格式（abstract algebra 子集）

评分方式有两种主流做法：

1. **直接概率比较（原始方式）**：不让模型生成文本，而是比较模型在位置上分别给 A/B/C/D 四个 token 的 log-probability，选概率最高的那个作为模型答案。这种方式不需要解析模型输出，但假设模型把答案“集中”在一个 token 上。
2. **生成式评估（chain-of-thought 方式）**：让模型先输出推理过程，最后提取答案字母。这种方式更接近真实使用，但需要正则匹配来抽取最终答案，也更容易受 prompt 格式影响。

同一道题，两种评分方式可能给出不同答案

直接概率比较和生成式评估的结果不一定一致。有些模型在 log-prob 上选对了，但让它生成推理过程反而会选错（反之亦然）。因此报告 MMLU 分数时，必须注明使用的评分方式。

为什么 MMLU 这类多选题格式有根本性局限

多选题把所有可能的答案限定在 4 个选项中，这意味着：(1) 随机猜测就有 25% 的基线；(2) 模型可以通过排除法而非真正理解来答题；(3) 真实场景中用户几乎不会给模型提供选项——他们期望开放式回答。更根本的是，多选格式把“知道答案”和“能生成答案”混为一谈。一个模型可能在看到正确答案时认出它，但如果要从零生成，它可能完全无法给出正确答案。这就像认脸和画脸是完全不同的能力。

4.3 MMLU-Pro

MMLU-Pro 做了几件事来提升难度：

- 去掉了一些噪声和过于简单的题。
- 把 4 选项扩成 10 选项。
- 用 chain-of-thought 评估，让模型多一点发挥空间。

它的意义在于提醒我们：一个基准一旦变得过于饱和，就不再能很好地区分模型。

4.4 GPQA 和 Humanity's Last Exam

GPQA 强调的是“Google-proof”，问题由博士级别参与者设计。Humanity's Last Exam 则更进一步，试图构造更难、更多模态、更多领域的题目，并通过多轮审核和 frontier model 过滤来降低平庸题目的比例。

基准为何会膨胀

基准题越容易，模型就越快饱和；题越难、越杂、越接近人类研究前沿，越能继续拉开模型差距。但难度提升的同时，构题成本、审核成本和题目质量问题也同步上升。

5 Instruction Following

ChatGPT 之后，很多人关注的不是“知识会不会答”，而是“你会不会照着我说的做”。这类任务更开放，也更接近真实产品交互。

5.1 Chatbot Arena

Chatbot Arena 的核心思想是 pairwise 比较：给用户两个匿名模型回答，让用户选更好的那个，然后基于这些对比算 ELO。它的优点是活的、可持续更新、并且能接纳新模型。

- 优点：接近真实用户偏好。
- 缺点：输入是活流量，比较不稳定，而且可能受到提示词分布影响。
- 结果：更像“谁更受欢迎”，不完全等于“谁更强”。

5.2 IFEval

IFEval 的策略是给 instruction 加上可自动检查的简单约束。这样可以保留一定的开放性，同时让评估可程序化。

IFEval 的思路

把自然语言指令拆成若干个可验证约束，比如长度、格式、包含某些关键词等。它不是在完整意义上评估语义正确性，而是在评估模型是否遵守了显式要求。

5.3 Chatbot Arena ELO 评分的计算直觉

Chatbot Arena 使用 Bradley-Terry 模型来从 pairwise 比较中推导出模型排名。其核心思想是：给每个模型赋一个“实力分” θ_i ，然后假设模型 i 赢模型 j 的概率为：

$$P(i \text{ beats } j) = \frac{1}{1 + 10^{(\theta_j - \theta_i)/400}}$$

这就是经典的 ELO 公式。直觉上：

- 如果两个模型分差为 0，则各有 50% 的胜率。
- 如果分差为 400，强者有约 91% 的胜率。
- 如果分差为 200，强者有约 76% 的胜率。

ELO 分差	强者预期胜率	直觉含义
0	50%	两个模型实力相当
100	64%	有优势但不压倒性
200	76%	明显更强
400	91%	几乎总赢
800	99%	代差级别的差距

表 2: ELO 分差与预期胜率的对对应关系

更新规则也很直觉：如果一个被认为很弱的模型赢了一个被认为很强的模型，赢家获得更多分数（因为这是一场“冷门”）；如果强者赢了弱者，分数变动很小。具体来说，每场比赛后的更新量为：

$$\Delta\theta_i = K \cdot (S_i - E_i)$$

其中 S_i 是实际结果（赢 =1，输 =0，平 =0.5）， E_i 是预期胜率， K 是学习率（控制排名变动的速度）。

ELO 的隐含假设与实际偏差

ELO 假设能力是一维的：如果 A 比 B 强，B 比 C 强，那么 A 一定比 C 强。但语言模型的能力是多维的——一个擅长代码的模型可能不擅长创意写作。这种“非传递性”在 Chatbot Arena 中确实存在，意味着 ELO 排名只是对复杂多维能力的一维投影。此外，用户群体的 prompt 分布也会影响排名：如果大部分用户在问编程问题，编程强的模型排名就会被高估。

5.4 AlpacaEval 和 WildBench

AlpacaEval 主要看 win rate，但一个重要问题是：评判者本身可能偏向某些模型。WildBench 则强调从真实人类对话中抽样，并尝试用更稳的判定方式来减少偏差。

LLM-as-judge 的风险

如果裁判模型和被评模型有共同偏差，或者裁判本身不稳定，那么 leaderboard 可能主要衡量的是“裁判喜欢什么”，而不是“用户真正需要什么”。

5.5 评估方法对比：人工、自动与 LLM-as-Judge

在 instruction following 和开放式生成领域，评估方法可以分为三大类。每种方法都有自己的适用场景和固有缺陷。

方法	优点	缺点	成本	代表
人工评估	最接近真实偏好；能捕捉细微语义	慢、贵、不可完全复现；标注者之间一致性有限	高	Chatbot Arena
自动 metric	快、便宜、完全可复现	只能检查表面特征；无法评估语义质量	低	IFEval, BLEU, pass@k
LLM-as-judge	速度与质量的折中；可大规模运行	存在位置偏差、长度偏差和自我偏好；与人类判断有系统性偏移	中	AlpacaEval, MT-Bench

表 3: 三种评估方法的系统性对比

LLM-as-judge 的具体偏差模式值得展开：

- **位置偏差**：在 pairwise 比较中，judge 倾向于选择第一个（或第二个）位置的回答，与内容无关。缓解方法是随机交换位置后取平均。
- **长度偏差**：更长的回答通常被 judge 评为更好，即使额外内容没有信息量。
- **自我偏好**：GPT-4 作为 judge 时会倾向于给 GPT-4 的回答更高分；Claude 作为 judge 也类似。
- **风格偏差**：更“礼貌”、更“结构化”的回答（用列表、标题等格式）通常得分更高，即使简洁的回答可能更有用。

5.6 主要 Leaderboard 对比

不同的 leaderboard 在设计哲学上有根本差异：

Leaderboard	评估方式	覆盖范围	更新频率	主要局限
HELM	标准化自动评估；统一 prompt 和 metric	多任务、多维度（准确率、鲁棒性、公平性、效率）	定期	任务选择偏保守；更新慢于 frontier
Chatbot Arena	真实用户 pairwise 投票	综合对话能力	实时	用户群体不均匀；prompt 分布偏向编程和英语
Open LLM Leaderboard	固定 benchmark 套件自动评估	开源模型为主；知识 + 推理任务	较快	容易被过拟合；benchmark 组合会变动

表 4: 三个主流 leaderboard 的设计哲学和适用场景

如何交叉使用多个 leaderboard

单个 leaderboard 就像从一个角度看一个多面体——你只能看到部分面。实际做模型选型时，建议至少交叉参考两到三个不同哲学的排行榜：一个看自动 benchmark（如 HELM 或 Open LLM Leaderboard），一个看人类偏好（如 Chatbot Arena），一个看实际部署指标（如 Artificial Analysis 的成本-性能前沿）。如果一个模型在三个维度上都排名靠前，比只在一个维度上领先更值得信赖。

6 Agent Benchmarks

一旦任务需要工具调用、长时推理或多轮迭代，传统单轮 benchmark 就不够了。于是出现了 agent benchmark。

6.1 SWE-bench

SWE-bench 要求模型面对真实 codebase 和 issue 描述，最后提交一个 PR，由单元测试来验证是否修复成功。它衡量的是实际软件工程能力，而不只是代码片段生成能力。

6.2 CyBench

CyBench 是安全或攻防相关的 CTF 任务，常用 first-solve time 等方式衡量难度和表现。它提醒我们：有些能力一旦做强了，既能用于正当防御，也能用于攻击。

6.3 MLEBench

MLEBench 更接近数据科学和机器学习工程任务：清洗数据、训练模型、调参、跑实验。它测的不仅是“会不会答题”，而是“会不会完成一个长流程任务”。

Agent benchmark 的特点

Agent benchmark 测的是 **模型 + 脚手架** 的整体效果。这样更接近真实产品，但也更难分清到底是模型本体改进，还是外部 scaffolding 起了作用。

7 Pure Reasoning

课程里讨论的下一个问题是：能不能把 reasoning 从 knowledge 里尽量分离出来？

7.1 ARC-AGI

ARC-AGI 试图测试更接近抽象推理的能力。它的吸引力在于：题目不太依赖背诵事实，而更依赖模式发现、概括和规则迁移。

- ARC-AGI-1 已经很难。
- ARC-AGI-2 更难，说明“纯推理”并不是一个容易孤立出来的维度。

7.2 MATH 和 AIME

MATH 数据集包含约 5000 道竞赛级数学题，从 1 级到 5 级难度。AIME (American Invitational Mathematics Examination) 则是美国数学竞赛的高水平题目，常被用来测试推理模型的上限。

评估数学题的关键技术细节：

- **答案归一化**：数学题的正确答案可以有多种等价形式（如 $\frac{1}{2}$ 、0.5、50%），评估时需要做符号级别的等价判断，而非简单的字符串匹配。
- **pass@k vs majority vote**：对于同一道题运行 k 次，pass@k 检查是否至少有一次答对，而 majority vote 取最多出现的答案。两种方式给出的“能力”含义不同——前者是“能不能做到”，后者是“稳不稳定”。
- **推理链的正确性**：模型可能给出正确答案但推理过程是错的（碰巧猜对），也可能推理过程合理但在最后一步算错。仅看最终答案的 metric 无法区分这两种情况。

推理和知识并不完全可分

现实模型的表现往往同时依赖知识、语言理解、注意力控制和搜索能力。所谓“纯推理”更多是一种评估理想，而不是一个完全纯净的实验条件。

7.3 本章小结

纯推理评估面临的核心困境是：推理能力很难完全与知识分离。ARC-AGI 类任务试图最小化知识依赖，但模型在这类任务上的进步往往依赖大量计算（搜索）和对模式的隐式“记忆”。MATH 类任务则需要数学知识和推理能力的结合，两者几乎不可能完全分开。

8 Safety Benchmarks

安全评估不是简单的“会不会拒答”。安全意味着很多东西：有害内容、越狱、双重用途、上下文相关的风险，以及不同地区法律与社会规范的差异。

8.1 HarmBench 与 AIR-Bench

HarmBench 关注违反法律或规范的有害行为；AIR-Bench 则从监管框架和公司政策出发，把风险做了更细的分类。它们都说明：安全不是单一指标，而是一组情境化的约束。

Benchmark	关注点	输入类型	评估方式
HarmBench	有害内容生成	对抗性 prompt（手写 + 自动生成）	分类器判断是否成功引出有害内容
AIR-Bench	监管与政策合规	基于真实法规分类的测试用例	按政策类别逐项评估
TruthfulQA	事实幻觉	容易引出常见错误的问题	人工标注的真实性判断
BBQ	社会偏见	歧义/消歧义的选择题	统计偏差方向和幅度

表 5: 主要安全 benchmark 的对比：侧重点不同，适用场景也不同

8.2 Jailbreaking

GCG 之类的方法可以自动优化 prompt，诱导模型绕过安全策略。这个现象说明：安全评估不能只依赖模型表面的拒答行为，还要考虑其是否真的抵抗攻击。

越狱攻击的演进大致分为几个阶段：

- **手工 prompt 注入**：早期通过角色扮演（“DAN”）、前缀注入等方式绕过安全策略。
- **自动化搜索（GCG）**：Zou et al. 提出了 Greedy Coordinate Gradient 方法，自动搜索能绕过安全策略的对抗性后缀。
- **多模态攻击**：通过图片、音频等非文本通道注入指令，绕过纯文本层面的安全过滤。
- **多轮渐进式攻击**：不在一个 prompt 中直接要求有害内容，而是通过多轮对话逐步引导模型偏离安全策略。

安全评估是动态攻防，不是静态打分

与知识类 benchmark 不同，安全评估面对的是主动对手。攻击手段在不断演进，因此“通过了当前的安全评估”不等于“安全”。模型发布后，必须持续监测新出现的攻击方式，并定期更新评估标准。这也是为什么部分安全研究机构主张在模型上线前做“红队压力测试”，而不是仅靠固定的 benchmark 套件。

8.3 Safety 评估中的 taxonomy 问题

安全评估最困难的部分之一是“什么算有害”本身没有统一定义。不同文化、法律体系和使用场景对“有害”的界定差异巨大：

- **法律差异**：某些内容在一个国家合法但在另一个国家违法（如仇恨言论在美国受第一修正案保护，但在德国是违法的）。
- **上下文依赖**：“如何制作炸弹”在化学教育语境和犯罪语境中完全不同。
- **程度问题**：从轻微不礼貌到严重的物理伤害指导，“有害”是一个连续谱而非二分类。

- **时间变化**：社会规范在演变，三年前被认为可以接受的回答，今天可能被视为不当。

为什么安全 benchmark 需要定期重做而不只是扩充

与知识 benchmark 不同，安全 benchmark 面临的不仅是“题目泄露”问题，还有“定义漂移”问题。随着社会规范和法规的变化，什么算“安全”的标准本身在移动。因此，安全评估套件不能只扩充题目，还需要定期审核和更新分类体系（taxonomy）本身。

8.4 Pre-deployment testing

课程提到美英安全研究机构在模型发布前参与测试。这里的关键不是某个具体流程，而是一个原则：在模型进入公众之前，先做更系统的安全审查。

8.5 什么是安全

安全既包含 **capability**，也包含 **propensity**。

- 一个模型可能有能力做坏事，但平时会拒绝。
- API 模型里，propensity 特别重要。
- 开源模型里，capability 更重要，因为安全层很容易被微调移除。

双重用途

CyBench 之类的任务既可以被看作安全评估，也可以被看作能力评估。一个系统越会做攻防，越需要明确它是在帮助防御，还是在增强攻击能力。

9 Realism

很多标准 benchmark 离真实使用场景非常远。现实用户不是在做考试，而是在“问问题”。

9.1 Quizzing vs Asking

1. **Quizzing**：用户知道答案，想测试系统。
2. **Asking**：用户不知道答案，想借助系统得到答案。

后者更接近真实产品使用，也更能体现模型的实际价值。

9.2 Clio 和 MedHELM

Clio 用模型分析真实用户数据，总结人们在问什么；MedHELM 则尝试把医疗评估从标准化考试转向更接近临床实践的任务集合。这里的核心矛盾是：越真实，越有价值，但越容易碰到隐私和合规问题。

真实性和隐私经常冲突

最接近真实流量的数据，往往也是最敏感、最难共享的数据。评估如果只看标准题，会偏离使用场景；如果只看真实数据，又可能受限于隐私和合规。

10 Validity

评估是否有效，是比”分数是多少”更底层的问题。

10.1 Train-test overlap

机器学习最基本的原则是不要在测试集上训练。可是在互联网语料时代，训练数据来自整个网络，是否污染很难彻底判断。

课程提到两条思路：

- **从模型侧推断污染**：利用数据点之间的可交换性等性质去估计重叠。
- **从流程侧要求披露**：让模型提供方报告 train-test overlap 或类似统计。

Benchmark Contamination: 数据泄露的系统性威胁

Benchmark contamination（基准污染）是当前大模型评估中最严重的系统性问题之一。由于预训练数据来自整个互联网，而许多 benchmark 的题目和答案也公开在网上，模型很可能在训练阶段就“见过”测试题。

具体的污染路径包括：

- **直接泄露**：benchmark 的题目和答案被收录进训练语料（如 Wikipedia 引用、学术论坛讨论、GitHub issue）。
- **间接泄露**：训练数据中包含关于 benchmark 的讨论、解题教程或排行榜分析。
- **合成数据放大**：用其他模型生成训练数据时，如果那些模型的训练也包含了 benchmark 数据，污染就通过合成数据间接传递。
- **评估集 rephrasing**：有些团队把评估题稍作改写后放入训练集，形式上不算直接泄露，但模型仍然能从中获益。

检测 contamination 的方法包括：检查 n-gram overlap、用 membership inference 判断模型是否“记住了”某个数据点、以及比较模型在公开题与未公开变体上的表现差异。但没有一种方法能做到完全可靠。

10.2 Goodhart's Law 与过拟合 Benchmark

Goodhart's Law: 当 benchmark 变成目标，它就不再是好的度量

“When a measure becomes a target, it ceases to be a good measure.” 这条经济学定律在 LLM 评估领域表现得尤为突出。

具体表现包括：

- **训练数据选择偏向**：如果知道模型会被 MMLU 评估，就有动机在训练数据中增加教科书和考试题的比例。
- **prompt engineering 过拟合**：为特定 benchmark 的 prompt 格式进行专门优化，使模型在该格式下表现远好于其他格式。
- **reward hacking**：在 RLHF 阶段，如果 reward model 与某些评估指标高度相关，模型会学会“讨好”评估指标而非真正提升能力。
- **系统级优化**：在 agent benchmark 中，团队可能为特定 benchmark 编写专门的 scaffolding 代码，使得分数反映的是工程投入而非模型能力。

Goodhart's Law 的深层含义是：评估永远只是真实目标的近似。一旦近似本身成为优化目标，它和真实目标的差距就会被放大。这也是为什么评估社区需要不断发布新的 benchmark：不仅因为旧的被饱和了，也因为旧的被优化废了。

10.3 Dataset quality

不是所有 benchmark 都是一开始就足够干净。比如 SWE-bench Verified 就是对原始 SWE-bench 做进一步修正得到的更可靠版本。更一般地，很多 benchmark 都需要通过人工或半自动方式不断做“platinum”级别的清洗。

数据集质量问题的常见类型：

质量问题	具体表现	对评估的影响
标签错误	参考答案本身就是错的	模型答对反而被判错，拉低得分
题目歧义	多个选项都可以被合理论证为正确	不同评估 prompt 给出不同结果
难度分布偏斜	大部分题太简单或太难	无法区分中等水平的模型
领域覆盖不均	某些子领域题目远多于其他	宏观分数被高频领域主导
时效性过期	题目基于过时的事实	模型学了更新的知识反而答“错”

表 6: 常见的 benchmark 数据集质量问题及其影响

评估有效性与数据质量

如果 benchmark 自身有错误、歧义或污染，那么模型排名的细微差异可能没有你想的那么可信。很多时候，改进 benchmark 和改进模型同样重要。

10.4 统计显著性：分差到底够不够大

即使 benchmark 数据质量没有问题，一个常被忽略的问题是：两个模型之间的分数差异是否具有统计显著性？

如果一个 benchmark 有 n 道题，模型准确率为 p ，那么准确率的标准误差近似为：

$$SE = \sqrt{\frac{p(1-p)}{n}}$$

以 MMLU 为例，如果模型准确率约为 85%，题目数约为 14000，则：

$$SE \approx \sqrt{\frac{0.85 \times 0.15}{14000}} \approx 0.003$$

这意味着 95% 置信区间大约是 $\pm 0.6\%$ 。因此，两个模型如果分差不到 1%，在统计上可能无法区分。但在 MMLU 的某些子领域（比如只有几十道题的小子集），标准误差会大得多，排名就更加不稳定。

评估的核心哲学：没有“唯一正确”的评估

这一讲最根本的信息不是某个 benchmark 好或不好，而是：**评估本质上是一个多目标、多视角、多利益方的近似问题**。不存在一个 benchmark 能对所有人、在所有场景下都给出“正确”答案。

好的评估实践不是追求一个完美的分数，而是：(1) 明确你的目标是什么；(2) 选择与目标最相关的度量；(3) 诚实地报告实验条件和局限；(4) 用多个互补的评估方式交叉验证。任何单一数字——无论是 MMLU 分数、ELO 排名还是 pass@k——都只是从一个角度看到的投影，不是全貌。

11 我们到底在评估什么

最后一个核心问题是：你到底在评估 model、system，还是 method？

11.1 方法还是系统

在早期机器学习里，人们更多评估的是 **方法**：给定标准数据集和标准训练流程，谁的算法更好。今天很多场景里，我们评估的是 **模型/系统**：只要最后用户体验更好，内部用了什么 prompt、工具或 scaffold 并不重要。

11.2 为什么这很重要

- 如果你在做研究，可能更关心方法是否普适、是否能推广。
- 如果你在做产品，可能只关心系统最终表现。
- 如果规则不明确，不同团队就会在不同层面“赢得”同一个 benchmark。

11.3 几个例外

课程也提到了一些仍然评估方法的例子：

- **nanoGPT speedrun**：固定数据，比较谁更快达到某个验证损失。

- **DataComp-LM**: 给定原始数据，按标准训练管线比较最终效果。

规则的游戏必须说清楚

一个好的评估一定要先声明：输入是什么、允许什么工具、是否允许额外训练、是测最终系统还是测单模型。否则分数本身没有可比性。

12 总结

这节课的结论可以概括成四句话：

- 没有唯一正确的评估方式，评估一定要跟目标对齐。
- 评估不是只看一个总分，要看具体样本和预测。
- 评估至少要同时考虑能力、安全、成本和真实性。
- 一定要讲清楚规则，是在评估方法，还是在评估模型/系统。

本课 takeaways

评估不是“找一个基准然后报分数”，而是一个围绕目标、输入、调用方式、输出和解释来设计的完整过程。真正好的评估，应该能回答你最开始想问的问题，而不是只给你一个看起来漂亮的数字。

13 把评估拆开看：定义、直觉、机制、应用、局限

前面已经把课程里出现的 benchmark 和问题铺开了。这里把它们重新整理成一个更稳定的分析框架，方便你以后面对一个新榜单时快速判断它到底在测什么。

13.1 定义：评估到底是什么

严格地说，评估是在一个给定问题设定下，把模型、系统或方法映射成可比较的数值或排序。这个映射必须先规定好四件事：

- 任务输入是什么。
- 模型如何被调用。
- 输出怎么被度量。
- 分数怎么被解释。

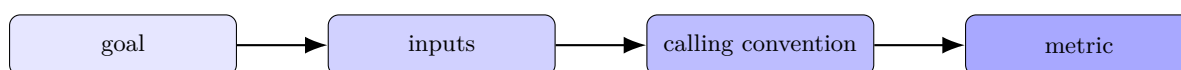


图 1: 评估不是一个分数，而是一条从目标到 metric 的定义链

定义层的判断标准

如果一套评估没有把目标、输入、调用和 metric 分开写清楚，那么它通常不适合直接拿来比较模型。它可能有参考价值，但不该被当成最终答案。

13.2 直觉：为什么评估看起来简单，做起来很难

直觉上，评估像是“把模型扔到题目里跑一遍”。实际问题是，语言模型不是一个固定函数，而是一个会被 prompt、工具、解码策略和上下文长度改变行为的系统。

看起来像	实际更像	后果
算分	系统设计	不同 prompt 会改变排名
测能力	选定场景下的能力投影	benchmark 不一定代表真实使用
看总分	看任务分解	单个分数会掩盖失败模式

表 7: 评估的直觉与现实之间的落差

为什么“看起来简单”是危险的

评估一旦被简化成“跑脚本”，就容易忽略污染、偏差、提示词敏感性和 judge bias。越是看起来规整的数字，越需要怀疑它背后的定义是否足够稳。

13.3 机制：一个完整评估管线怎么运作

可以把现代评估管线理解成五步：

1. 选择任务集合。
2. 设计输入格式和 prompt。
3. 运行模型或系统。
4. 收集输出并计算指标。
5. 聚合结果并解释。

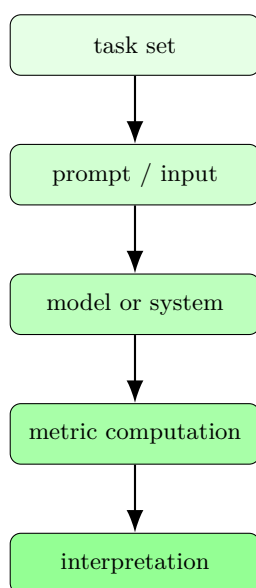


图 2: 评估管线的五个环节：任务、输入、系统、指标、解释

机制层最常见的误解

很多争论其实不是“哪个模型更强”，而是“哪个评估管线更公平”。如果任务、prompt 或 judge 改了，结果就可能改，甚至改得比模型差异还大。

13.4 应用：什么时候该用什么评估

不同场景应该用不同评估。

场景	应该更关心什么	更合适的评估
买模型 / 选产品	用户体验、价格、稳定性	arena、成本前沿、真实流量
做基础研究	可解释的能力增长	benchmark + ablation + scaling law
做安全审查	有害行为与越狱	curated safety sets, red teaming
做系统优化	吞吐、延迟、成本	throughput / latency / cost profiling

表 8: 评估方法必须随目标变化，而不是一套分数走天下

如何快速选 benchmark

先问自己：我是在测知识、指令遵循、工具使用、推理，还是安全风险？如果问题没想清楚，先别急着选分数，先选任务定义。

13.5 局限：为什么没有一个 benchmark 能包打天下

任何 benchmark 都至少有四类局限：

- **覆盖不全**：真实场景总比 benchmark 更杂。
- **可被游戏化**：一旦分数成为目标，就会有人围着分数优化。
- **会过时**：数据集一旦饱和，区分力就下降。
- **会偏置**：人类判断、LLM judge、流量数据都可能带偏。

局限不是 bug，而是事实

评估本身就是近似。你不是在寻找一个绝对真理，而是在寻找一个足够稳、足够相关、足够可解释的代理指标。

14 Worked Example 1: 一个 toy perplexity 计算

为了把 perplexity 讲透，我们做一个非常小的例子。假设一个模型对两个 token 的条件概率分别是：

$$p(x_1) = 0.5, \quad p(x_2|x_1) = 0.25.$$

那么这两个 token 的联合概率是

$$p(D) = 0.5 \times 0.25 = 0.125.$$

如果数据集长度是 2，那么 perplexity 就是

$$\text{PPL}(D) = \left(\frac{1}{0.125} \right)^{1/2} = \sqrt{8} \approx 2.83.$$

这意味着，模型平均上相当于在“每一步从大约 2.83 个等可能选择中猜一个”。

这个例子的意义

perplexity 不是抽象术语，它就是把序列概率转成更直观的“平均分叉度”。数值越小，模型对真实文本越不惊讶。

toy 例子和真实评估的差异

真实场景里，模型面对的不是两个 token，而是数百到数万 token；概率也不是均匀的，所以 perplexity 的可解释性会和任务难度、上下文长度、tokenization 共同耦合。

15 Worked Example 2: 同一模型，三种评估会给出三种结论

假设一个模型有三个特征：

- 知识题做得不错。
- 对开放式对话回答流畅。
- 价格比同类模型低。

如果你把它放在不同的评价框架里，结论会明显不同：

评估方式	你会看到什么	可能结论
MMLU / GPQA	知识覆盖与推理能力	“能力不错”
Chatbot Arena	人类偏好、回复风格	“用户喜欢它”
Artificial Analysis	能力-价格前沿	“性价比高”

表 9: 同一模型在不同评估框架下会得到不同结论

这不是矛盾，而是视角不同

一个模型“在知识题上强”并不自动推出“用户体验最好”，更不自动推出“安全或最便宜”。评估本来就应该多维度并行。

16 Worked Example 3: Chatbot Arena 为什么用 pairwise

Chatbot Arena 不直接让模型做标准题，而是让用户在两个匿名回复之间做选择。这样做的好处是，用户不需要把好坏翻译成一个抽象分数，只要回答“哪个更好”。

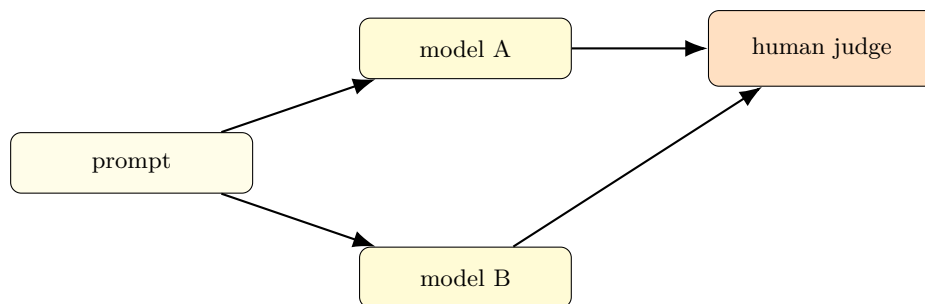


图 3: Chatbot Arena 的核心: pairwise comparison 比绝对打分更自然

它的一个关键优势是，只需要比较两个答案，不需要人类定义“这次到底是 7 分还是 8 分”。但它也有明显问题：

- 输入分布是活流量，不稳定。
- 用户群体不是均匀抽样。
- ELO 是相对排名，不等于绝对能力。

为什么 pairwise 很常见

pairwise 比绝对评分更容易做出一致判断，也更接近真实用户的“我更喜欢 A 还是 B”。缺点是你只能得到相对顺序，解释空间也更依赖流量和对局采样。

17 Worked Example 4: 安全评估和能力评估为什么会混在一起

考虑一个能写 exploit 的模型。它有两种可能状态：

- 能力很强，但默认拒答。
- 能力很强，而且愿意输出有害内容。

对 API 产品来说，前者和后者差异极大，因为用户实际上能看到的是 **propensity**。对开源模型来说，用户可以轻松改掉安全层，因此 **capability** 又变得更关键。

视角	关心什么	为什么
API 产品	propensity	模型实际会不会输出有害内容
开源模型	capability	安全策略能否被轻易移除
监管机构	风险类别	是否符合政策、法律和部署要求

表 10: 安全评估里 capability 和 propensity 是不同维度

不要把“会做坏事”和“会不会说坏话”混为一谈

有些评估在测模型的潜在能力，有些在测它在当前部署配置下的外显行为。两者都重要，但不能混淆。

18 一个更完整的 benchmark 选择指南

如果你要给一个新模型选评估方案，可以按下面的顺序问：

1. 我想测知识、推理、对话、工具、还是安全？
2. 我关心的是模型本身，还是整个系统？
3. 我需要绝对分数，还是相对排序？
4. 我在乎成本吗？
5. 我能否接受真人 judge 或 LLM judge？
6. 我是否担心污染、过拟合或 benchmark 饱和？

一个实用原则

先用最小代价找到能回答问题的评估，再去追求更复杂的评估。不要反过来。

评估设计的工程感

评估不是收集分数，而是设计实验。像做实验一样，你要控制变量、定义假设、记录条件，并且诚实地汇报失败和限制。

19 更深一点的局限：为什么榜单会失真

这一讲反复提醒 “evaluation crisis”，原因就是榜单会慢慢失真。

19.1 污染与饱和

一旦训练数据和测试数据有重叠，benchmark 就不再纯净。另一方面，数据集越简单，模型越快饱和，榜单越不能区分模型。

19.2 奖励错配

如果 leaderboard 奖励的是某种局部优势，比如固定格式、特定题型或短期偏好，模型开发就会围着这些奖励转，最后不一定更有用。

19.3 judge bias

LLM-as-judge 或 human judge 都可能偏向某些风格。比如更长、更流畅、更像“礼貌助手”的答案，未必真的更正确。

榜单不能替代理解

榜单只能告诉你“这个系统在某些定义下表现更好”，不能自动告诉你“它为什么好”或“它在你关心的真实任务里也一定更好”。

20 Worked Examples: 评估如何真的算出来

这一讲最容易停留在概念层。为了避免这种情况，这里把几个最常见的评估任务拆成具体算例。

20.1 例 1: 多任务知识评估怎么汇总

假设你有三个子任务，每个任务都是 10 道题，模型分别答对 8、6、9 题。最直接的 micro average 是

$$\frac{8 + 6 + 9}{30} = 0.77.$$

如果改成按任务平均的 macro average，则是

$$\frac{0.8 + 0.6 + 0.9}{3} = 0.766\bar{6}.$$

这两个数接近，但当每个子任务样本数不同的时候，它们会明显不一样。

子任务	题数	答对	准确率
task A	10	8	80%
task B	10	6	60%
task C	10	9	90%

表 11: 一个简单的多任务汇总例子

macro 和 micro 的差异

micro average 让大任务占更大权重，macro average 让每个任务权重相同。一个偏向“总体通过率”，一个偏向“任务均衡性”。

20.2 例 2: pairwise 胜率怎么理解

假设某个模型在 20 场对决里赢了 13 场，输 7 场，那么原始 win rate 是 65%。但这不是完整信息，因为：

- 对手是谁很重要。
- 哪些 prompt 被抽中很重要。
- 置信区间有多宽也很重要。

如果你把 pairwise 结果想成二项分布，那么一个粗略的标准误差可以写成

$$\sqrt{\frac{p(1-p)}{n}},$$

其中 $p = 0.65, n = 20$ 。这说明样本还不算大，波动可能不小。

ELO 不是神谕

ELO 的价值是把一堆 pairwise 关系压缩成一个可比较排序，但它仍然依赖采样分布、比较对象和更新规则。排行榜看起来是一个数，实际是一整套实验设计。

20.3 例 3：安全评估里的混淆矩阵

安全任务经常可以被写成一个二分类问题：模型的响应要么安全，要么不安全。此时就会有四种情况：

- 真安全且被判安全。
- 真安全但被误判不安全。
- 真不安全但被放过。
- 真不安全且被正确拦截。

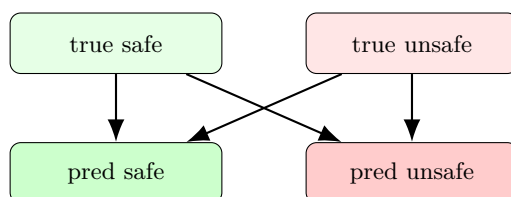


图 4：安全评估可以被组织成一个混淆矩阵问题，但真正的难点在于标签定义和上下文依赖

安全类指标最容易出错的地方

把“拒答率高”误认为“安全性高”。如果模型过度拒答，它可能更安全，也可能只是更没用；如果模型过度配合，它可能更好用，也可能更危险。必须看任务定义。

20.4 例 4：模型研究和产品决策看的是不同东西

如果你是研究者，往往关心一个改动是否真正提升了能力边界；如果你是产品经理，你可能更关心用户留存、价格和响应速度。

角色	主要目标	常用评估
researcher	可发表、可复现、可外推的改进	benchmark, ablation, scaling law
product builder	更好的实际体验与成本控制	arena, traffic, latency, cost
policy/safety team	风险可控、可审计	curated safety suites, red teaming

表 12：同一模型在不同组织里的评估目标并不一样

为什么这一点重要

如果评估目标混淆，团队会把资源投到错误的优化方向上。研究上的胜利不一定是产品上的胜利，产品上的胜利也不一定是安全上的胜利。

21 Benchmark Families: 每一类 benchmark 在测什么

可以把本讲提到的 benchmark 大致归成五类。这样你以后看新榜单时，就知道它在整个地图上的位置。

家族	测什么	常见代表	常见盲点
knowledge	事实、学科知识	MMLU, GPQA	容易被污染和饱和
instruction following	符合指令	IFEval, AlpacaEval	judge bias、prompt 敏感
agent	工具和长流程	SWE-bench, MLEBench	scaffolding 影响大
reasoning	抽象推理	ARC-AGI	知识和推理难分离
safety	有害行为与拒答	HarmBench, AIR-Bench	情境依赖、地区依赖

表 13: 五类 benchmark 的位置感：测的东西不同，失真方式也不同

21.1 主要 Benchmark 详细对比

下面是一张更细致的对比表，包含每个 benchmark 的规模、评估方式和代表性模型得分参考（截至 2025 年中）。注意：分数会随模型版本和评估细节变化，这里只给出数量级和相对关系。

Benchmark	测量能力	题目数	评估方式	代表性分数参考
MMLU	57 学科知识	~14k	多选题 log-prob 或生成式	GPT-4: ~86%, Llama 3 70B: ~79%
MMLU-Pro	更难知识	~12k	10 选项 + CoT	比 MMLU 低约 15–20 个百分点
GPQA	博士级知识	~450	多选题	GPT-4: ~39%, 人类专家: ~65%
IFEval	指令遵循	~500	自动约束检查	前沿模型: 70–85%
Chatbot Arena	综合偏好	持续增长	人类 pairwise	ELO 排名（相对分数）
SWE-bench	软件工程	~2.3k	单元测试通过率	前沿 agent 系统: 30–50%
MATH	数学推理	~5k	精确匹配	GPT-4: ~52%, 推理模型: 80+%
HumanEval	代码生成	164	pass@k	GPT-4: ~87%, 开源模型: 50–75%
ARC-AGI	抽象推理	~400	精确匹配网格	前沿模型 [带搜索]: ~50%
HarmBench	安全性	~500	分类器判断	因攻击方式差异大，不宜直接比较

表 14: 主要 benchmark 详细对比——规模、评估方式和参考分数

先判断家族，再判断分数

如果你连一个 benchmark 属于哪一类都没判断清楚，就直接比较分数，通常只会得到看似精确但实际无意义的结论。

如何把 benchmark 家族和问题匹配起来

问“这个 benchmark 的失败，是否真的会影响我在乎的结果？”如果答案是肯定的，它才值得作为主指标；否则它最多是辅助信号。

22 如何写一个靠谱的 evaluation report

如果评估不是只看一个数字，那报告也不应该只写一个数字。一个好的 evaluation report 至少应该说明：

- 任务定义和输入来源。
- 调用方式和任何 scaffolding。
- 指标定义、聚合方式和样本数量。
- 置信区间、方差或重复运行结果。
- 明确的局限与已知失败模式。

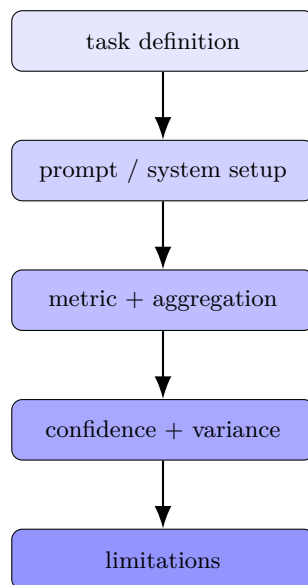


图 5: 一个靠谱的 evaluation report 应该显式写出实验条件，而不是只给单个分数

什么叫“报告写完整了”

不是把所有数字都堆出来，而是让读者知道这个数字是在什么规则下得到的、它能否复现、它对你的目标是否真的有意义。

22.1 置信区间不是装饰

如果样本量很小，单次分数的波动可能很大。尤其是 pairwise 或少样本任务，最好报告重复实验或置信区间，而不是把某一次运行当成定论。

如何读置信区间

如果两个模型分数差距很小，而误差条大量重叠，那“领先”未必有实际意义。报告里如果完全不提方差，通常说明作者希望你忽略不确定性。

最常见的报告错误

只报平均值，不报样本数；只报总体分数，不报失败案例；只报正结果，不报对照组。这些都会让评估看起来比它实际更稳。

22.2 一个最小检查清单

拿到一份新的 benchmark 结果时，你至少可以快速检查这几项：

1. 它测的是知识、推理、对话、工具还是安全？
2. 它是测模型、系统还是方法？
3. 输入和 prompt 是否固定？
4. 输出是否依赖 judge？
5. 有没有重复运行和方差？
6. 有没有已知污染或饱和问题？

读 benchmark 的底线

如果这些问题答不出来，就不要急着根据分数下结论。先把实验条件搞清楚，再看模型到底改进了什么。

23 总结与延伸

23.1 本章小结

把这一讲压缩成最后几条经验法则，就是：

- 评估首先是目标定义问题。
- 不同目标对应不同 benchmark。
- 绝对分数很重要，但相对排序、成本和鲁棒性同样重要。
- 任何 benchmark 都有局限，尤其是在数据污染、饱和和 judge bias 上。
- 真正好的评估要能推动模型开发，而不是只让表格更好看。

最后一句更实用的话

当你看到一个很漂亮的 leaderboard，先别急着相信分数，先问：它到底在测什么，它没测什么，它是模型榜，还是系统榜，还是方法榜？